



Git Essential Commands

Introduction to Git

Overview:

Git is a tool used to keep track of changes to files, especially the code of the projects. It is termed a distributed version control system because of its behaviour to allow multiple people to work on the same project, even if they are not connected to a common server. It keeps track of every little change or update you make, you can always look back at previous versions or undo mistakes if needed.

Understanding Git

Key Concepts:

- **Repository (Repo):** Directory with project files and change history.
 - **Commit:** Snapshot of your project at a specific time.
 - **Branch:** Separate line of development within a repository.
 - **Merge:** Combining changes from different branches.
 - **Remote:** Online version of your project for collaboration.
-

Basic setup and starting a new repo

- **git init:** Initializes a new Git repository.

```
git init
```

- **git clone [url]:** Clones a remote repository locally.

```
git clone https://github.com/username/repo.git
```

- **git config:** Used to view or set Git configuration settings, like your name, email, editor, aliases, and more.

```
git config --global user.name "Your Name"
```

```
git config --global user.email "you@example.com"
```

Tracking changes and commit

- **git status:** Displays the state of your working directory.

```
git status
```

- **git add [file]:** Stages file changes for commit.

```
git add . # adds all changes in the current directory
```

- **git commit -m "[message]":** Records a snapshot with a message.

```
git commit -m "Add new feature"
```

Branching and Merging

- **git branch:** Lists or creates branches.

```
git branch # Lists branches
```

```
git branch new-feature # Creates a new branch
```

- **git checkout :** Switches your working directory to a different branch or restores files to a specific state.

```
git checkout new-feature # switches to different branch.
```

```
git checkout -- file.txt # discard changes in 'file.txt'
```

- **git merge [branch]:** Merges a branch into the current branch.

```
git merge new-feature
```

Remote Repositories

- **git remote -v:** Lists associated remote repositories.

```
git remote -v
```

```
git remote add origin # used to link your local Git repository to a remote repository.
```

- **git push [remote] [branch]:** Updates remote with local commits.

git push origin main

- **git pull [remote] [branch]:** Integrates remote changes locally.

git pull origin main

Viewing History and Changes

- **git log:** Shows commit history.

git log --oneline # shows the commit history in a compact, single-line format.

- **git diff:** Displays changes between commits or branches.

git diff # shows the line-by-line changes in your files that are not yet staged.

git diff --staged # changes that are staged (after `git add`)

- **git show:** Displays detailed information about a specific commit, including changes made, author, and date.

git show abc1234 # show a specific commit by its hash

Undoing Changes

- **git restore:** undo changes in your working directory or staging area.

git restore <file> # replaces the file with the last committed version.

git restore --staged <file> # removing it from the staging area but keeping your changes in the working directory.

- **git revert:** Creates a new commit that undoes the changes introduced by the latest commit (HEAD).

git revert <commit-id> # reverts a commit by applying the opposite changes as a new commit.

- **git reset:** It's a powerful command for moving the HEAD, changing what's staged, or even deleting commit history (with caution).

git reset --soft # moves HEAD to a commit, keeps staged and working directory changes

git reset --mixed # moves HEAD, unstages changes, keeps working directory changes

git reset --hard # moves HEAD, discards staged and working directory changes

Relevant Resources

<https://git-scm.com/doc>

Here, you'll find comprehensive documentation and references on Git. Feel free to explore and deepen your understanding.